

AIAC LAB_ASSIGNMENT_7.4

NAME :- CH.NAKSHATRA

HALL TICKET NO : - 2303A51094

BATCH NO :- 02

Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

Bug: Mutable default argument

```
def add_item(item, items=[]):
```

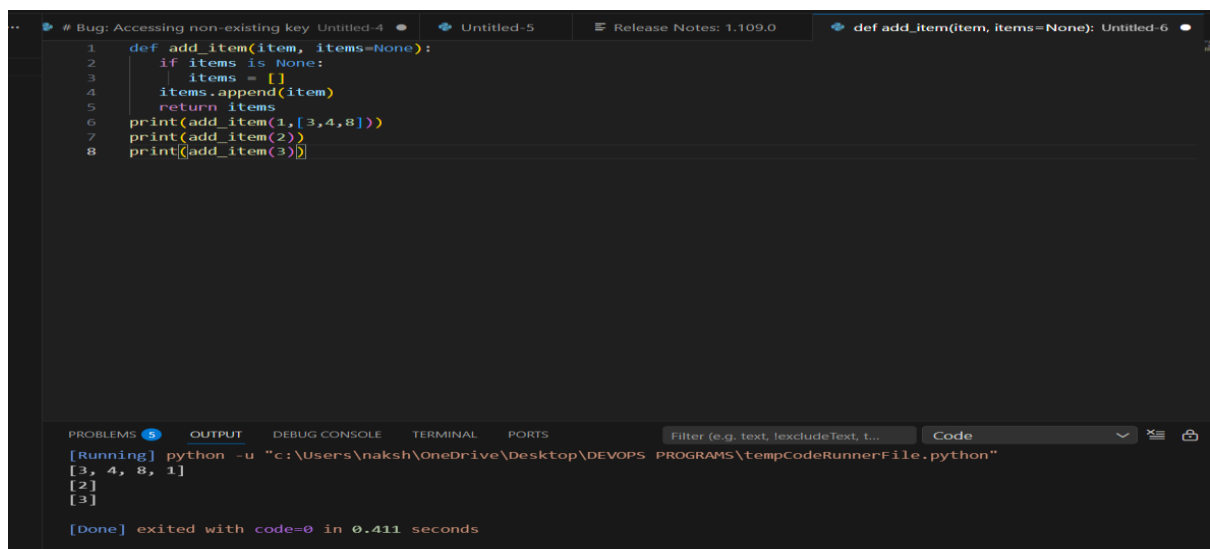
```
    items.append(item)
```

```
    return items
```

```
print(add_item(1))
```

```
print(add_item(2))
```

Expected Output: Corrected function avoids shared list bug.



The screenshot shows a code editor with a Python function `add_item` that has a mutable default argument `items=[]`. The function appends the item to the list and returns it. The code is executed, and the output shows the function being called multiple times, demonstrating the bug where the list is shared across calls. The output is as follows:

```
[Running] python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCodeRunnerFile.python"
[3, 4, 8, 1]
[2]
[3]
[Done] exited with code=0 in 0.411 seconds
```

Task 2 (Floating-Point Precision Error)

Task: Analyze given code where floating-point comparison fails.

Use AI to correct with tolerance.

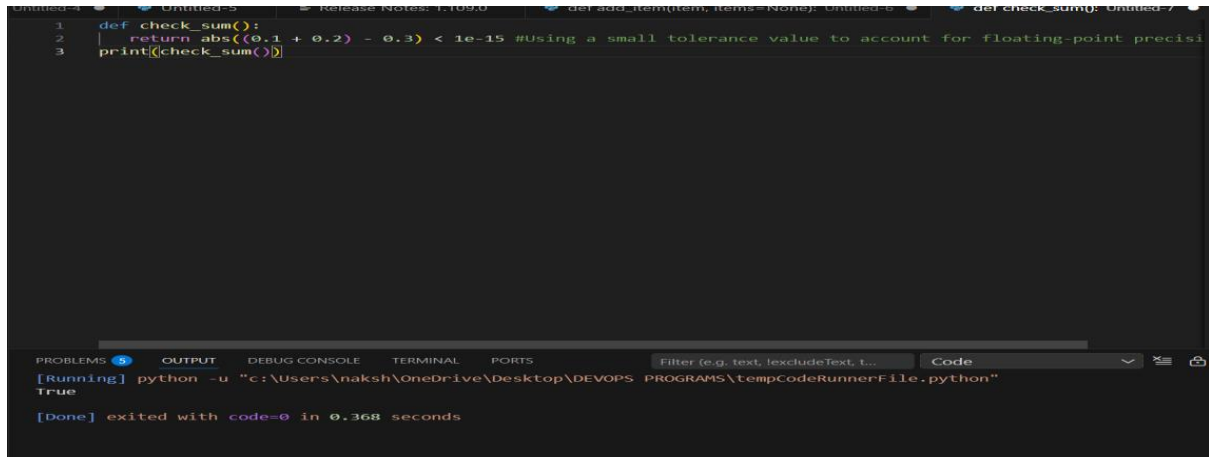
Bug: Floating point precision issue

```
def check_sum():
```

```
    return (0.1 + 0.2) == 0.3
```

```
print(check_sum())
```

Expected Output: Corrected function

A screenshot of a code editor interface. The editor shows a Python function named 'check_sum()' which returns the result of the expression '(0.1 + 0.2) == 0.3' wrapped in an 'abs()' function. A comment indicates a tolerance value of 1e-15 is used. The function is then printed. Below the code, the terminal output shows the command 'python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCodeRunnerFile.python"' being executed, resulting in the output 'True' and a completion message '[Done] exited with code=0 in 0.368 seconds'.

Task 3 (Recursion Error – Missing Base Case)

Task: Analyze given code where recursion runs infinitely due to missing base case. Use AI to fix.

Bug: No base case

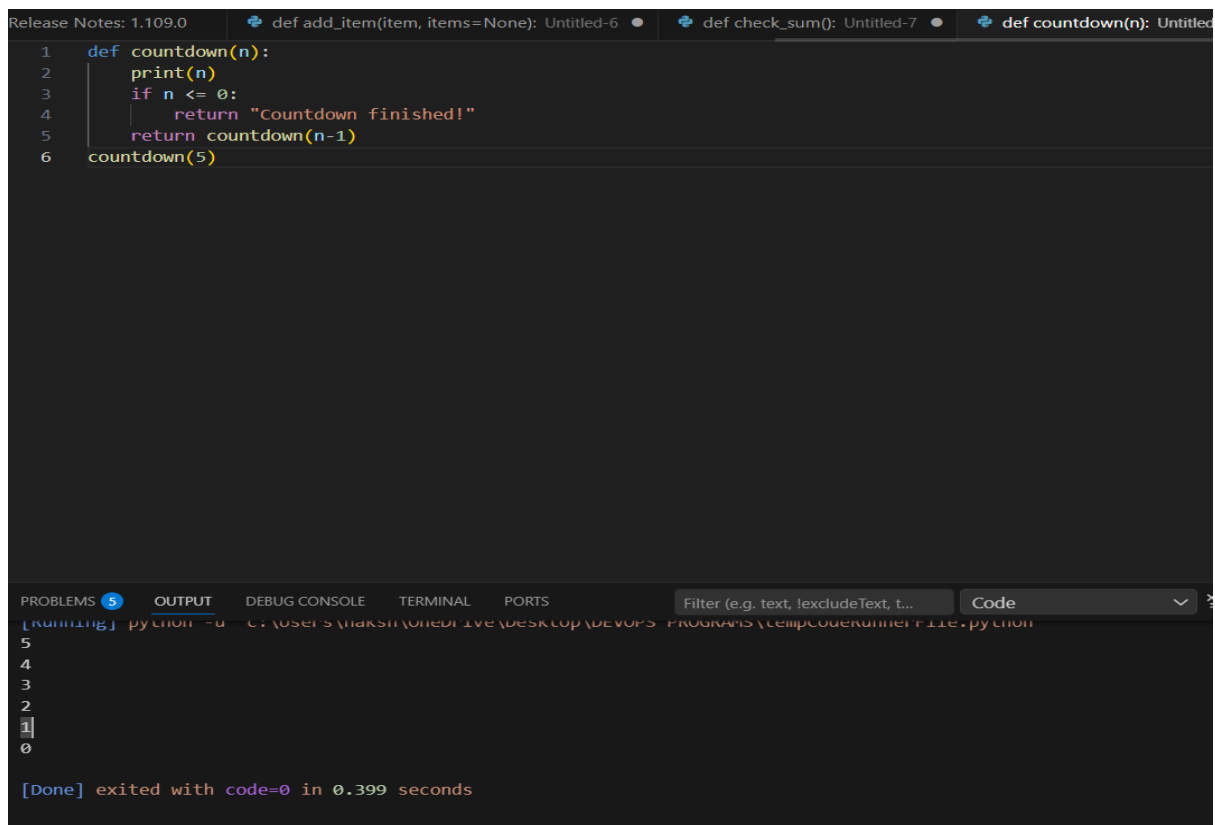
```
def countdown(n):
```

```
    print(n)
```

```
    return countdown(n-1)
```

```
countdown(5)
```

Expected Output : Correct recursion with stopping condition.



The screenshot shows a code editor with three tabs: 'def add_item(item, items=None): Untitled-6', 'def check_sum(): Untitled-7', and 'def countdown(n): Untitled-8'. The active tab 'def countdown(n): Untitled-8' contains the following Python code:

```
1 def countdown(n):
2     print(n)
3     if n <= 0:
4         return "Countdown finished!"
5     return countdown(n-1)
6 countdown(5)
```

Below the code editor, the 'OUTPUT' panel shows the execution results:

```
[running] python -u C:\Users\maxst\OneDrive\Desktop\DEVOPS\PROGRAMS\TempCodeRunner\File.py
5
4
3
2
1
0
[Done] exited with code=0 in 0.399 seconds
```

Task 4 (Dictionary Key Error)

Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

Bug: Accessing non-existing key

def get_value():

data = {"a": 1, "b": 2}

return data["c"]

print(get_value())

Expected Output: Corrected with .get() or error handling.

```
1 def get_value():
2     Click to add a breakpoint | 1, "b": 2}
3     try:
4         return data["c"]
5     except KeyError:
6         return "key not found"
7     print(get_value())
```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, lexcludeText, t... Code

[Running] python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCodeRunnerFile.python"

key not found

[Done] exited with code=0 in 0.37 seconds

Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect and fix it.

Bug: Infinite loop

def loop_example():

i = 0

while i < 5:

print(i)

Expected Output: Corrected loop increments i.

```
1 def loop_example():
2     i = 0
3     while i < 5:
4         print(i)
5         i += 1
6     loop_example()
```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, lexcludeText, t... Code

[Running] python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCodeRunnerFile.python"

0
1
2
3
4

[Done] exited with code=0 in 0.402 seconds

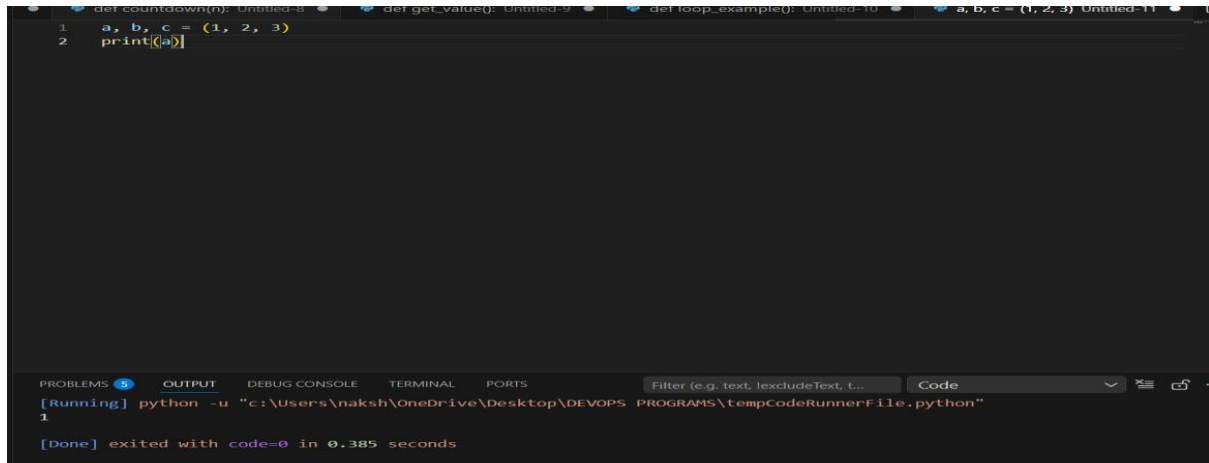
Task 6 (Unpacking Error – Wrong Variables)

Task: Analyze given code where tuple unpacking fails. Use AI to fix it.

Bug: Wrong unpacking

a, b = (1, 2, 3)

Expected Output: Correct unpacking or using _ for extra values.



The screenshot shows a code editor with a Python script. The script has two lines: `1 a, b, c = (1, 2, 3)` and `2 print(a)`. The output panel shows the result of running the script: `[Running] python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCodeRunnerFile.py"` followed by `1` on the next line. The status bar indicates `[Done] exited with code=0 in 0.385 seconds`.

Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

Bug: Mixed indentation

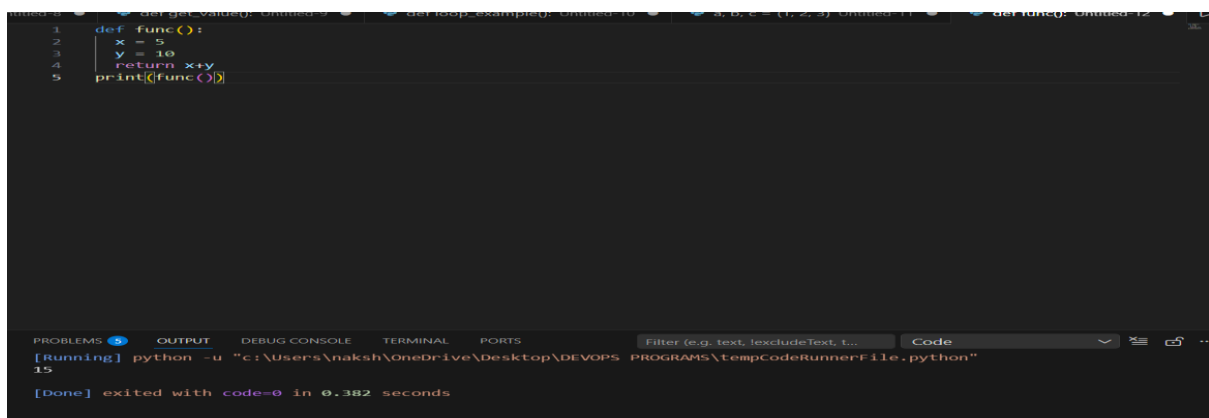
def func():

x = 5

y = 10

return x+y

Expected Output : Consistent indentation applied.



The screenshot shows a code editor with a Python script. The script has five lines: `1 def func():`, `2 x = 5`, `3 y = 10`, `4 return x+y`, and `5 print(func())`. The output panel shows the result of running the script: `[Running] python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCodeRunnerFile.py"` followed by `15` on the next line. The status bar indicates `[Done] exited with code=0 in 0.382 seconds`.

Task 8 (Import Error – Wrong Module Usage)

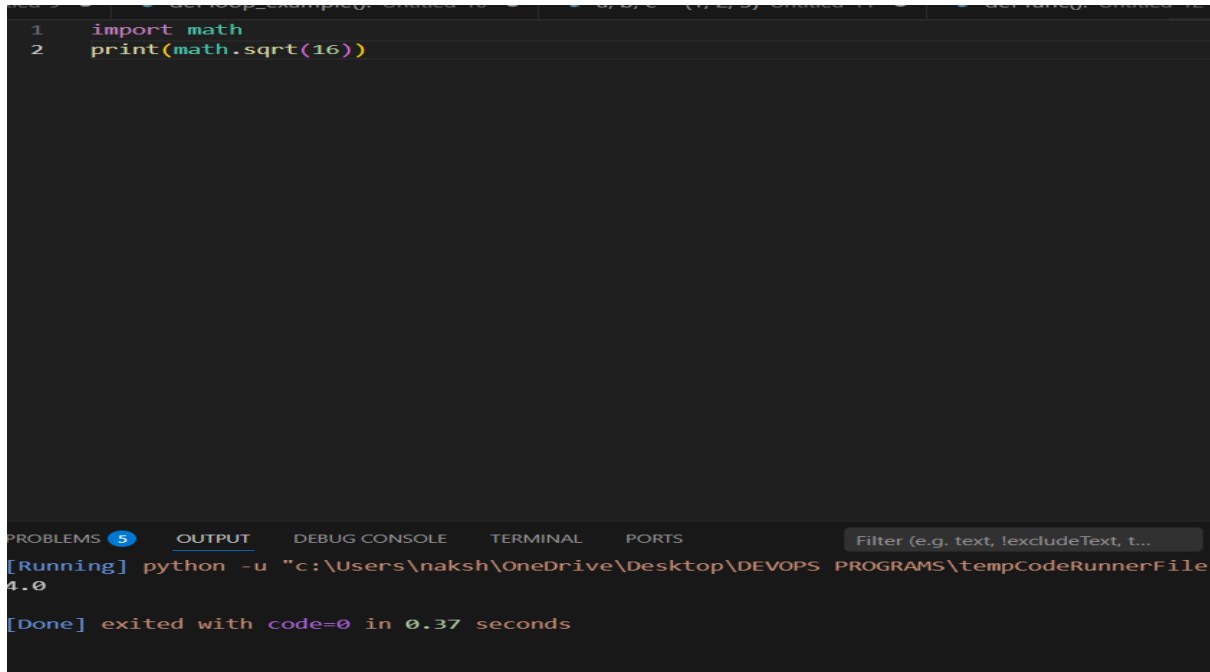
Task: Analyze given code with incorrect import. Use AI to fix.

Bug: Wrong import

```
import maths
```

```
print(maths.sqrt(16))
```

Expected Output: Corrected to import math



The screenshot shows a code editor with two lines of Python code: `1 import math` and `2 print(math.sqrt(16))`. Below the code editor, the 'OUTPUT' tab is selected, showing the command `python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCodeRunnerFile` and the output `4.0`. The status bar at the bottom indicates `[Done] exited with code=0 in 0.37 seconds`.

Task 9 (Unreachable Code – Return Inside Loop)

Task: Analyze given code where a return inside a loop prevents full iteration. Use AI to fix it.

Bug: Early return inside loop

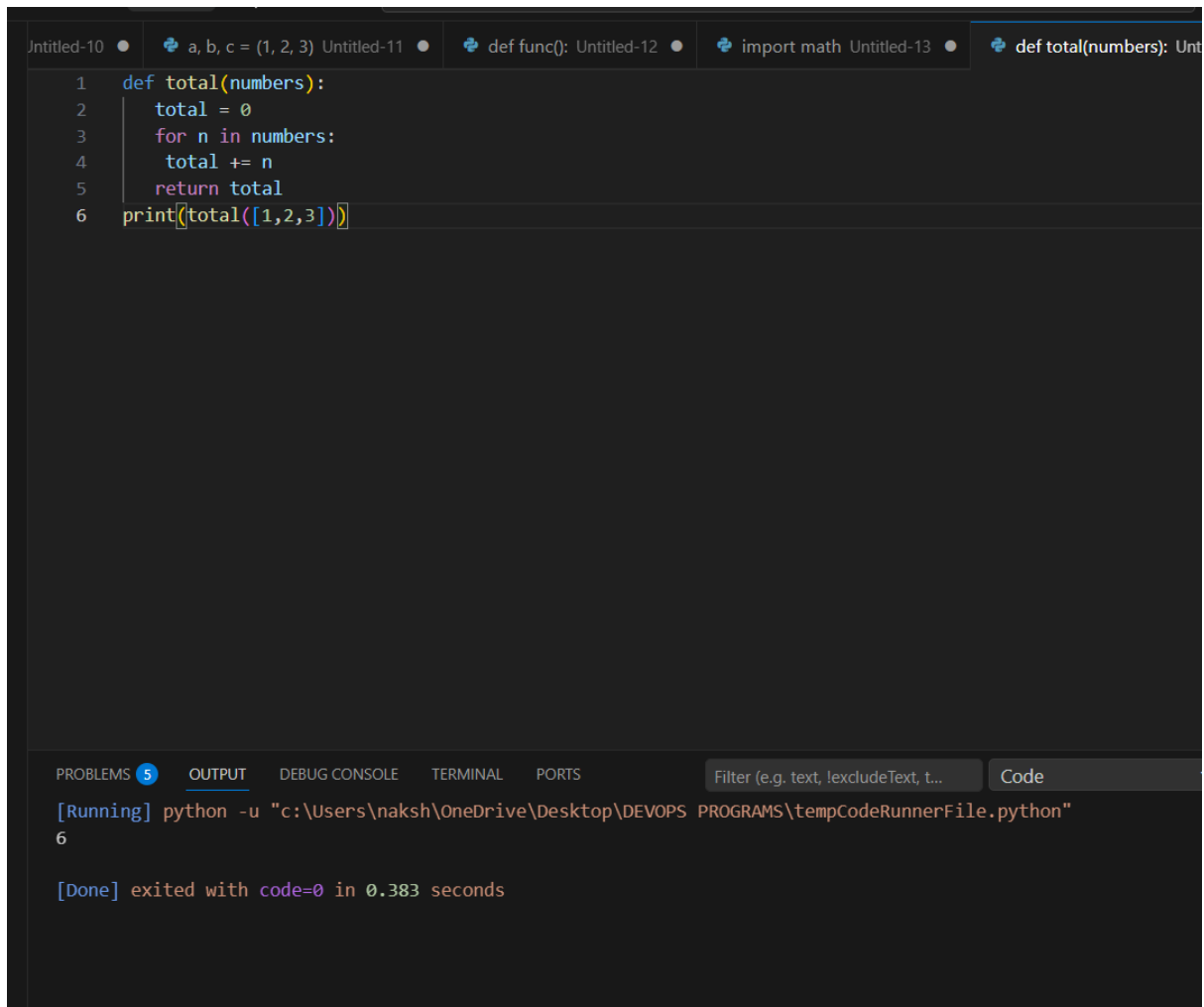
```
def total(numbers):
```

```
    for n in numbers:
```

```
        return n
```

```
print(total([1,2,3]))
```

Expected Output: Corrected code accumulates sum and returns after loop.



The screenshot shows a code editor with several tabs. The active tab contains the following Python code:

```
1 def total(numbers):
2     total = 0
3     for n in numbers:
4         total += n
5     return total
6 print(total([1,2,3]))
```

Below the code editor, the 'OUTPUT' tab is selected, showing the execution of the code:

```
[Running] python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCodeRunnerFile.python"
6

[Done] exited with code=0 in 0.383 seconds
```

Task 10 (Name Error – Undefined Variable)

Task: Analyze given code where a variable is used before being defined. Let AI detect and fix the error.

Bug: Using undefined variable

```
def calculate_area():
    return length * width
print(calculate_area())
```

Requirements:

- Run the code to observe the error.
- Ask AI to identify the missing variable definition.
- Fix the bug by defining length and width as parameters.
- Add 3 assert test cases for correctness.

Expected Output :

- **Corrected code with parameters.**
- **AI explanation of the bug.**

Successful execution of assertions

```

1 def calculate_area(length, width):
2     return length * width
3 print([calculate_area(5,3)])

```

OUTPUT

[Running] python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCodeRunnerFile.python"

15

[Done] exited with code=0 in 0.351 seconds

Task 11 (Type Error – Mixing Data Types Incorrectly)

Task: Analyze given code where integers and strings are added incorrectly. Let AI detect and fix the error.

Bug: Adding integer and string

```

def add_values():
    return 5 + "10"
print(add_values())

```

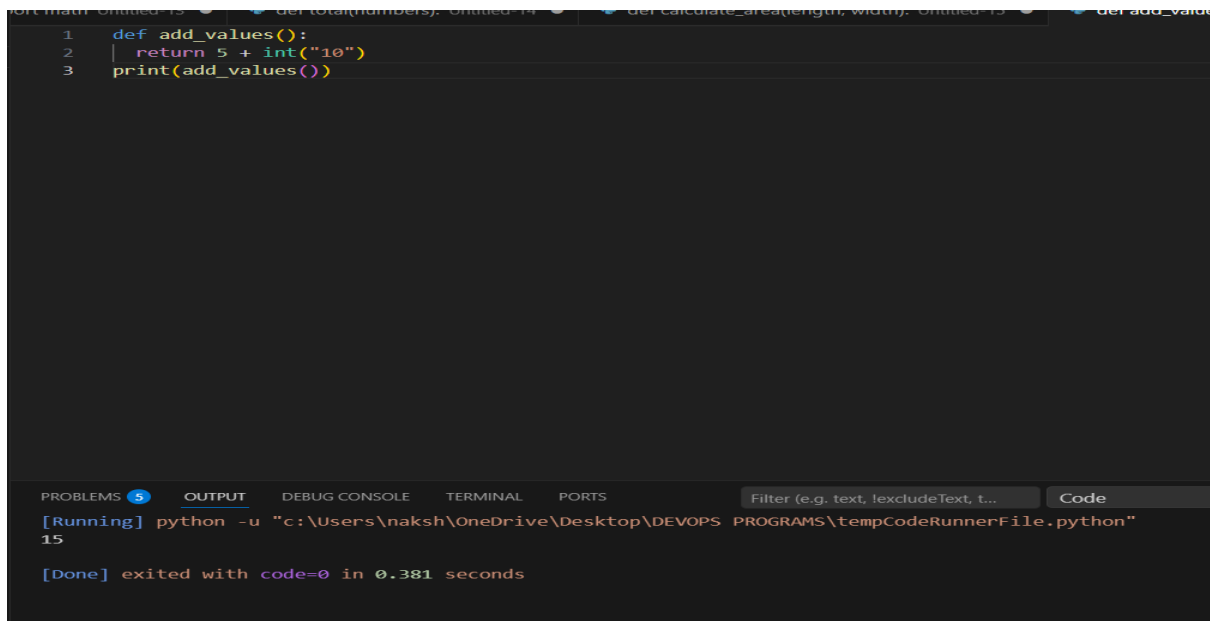
Requirements:

- **Run the code to observe the error.**
- **AI should explain why int + str is invalid.**
- **Fix the code by type conversion (e.g., int("10") or str(5)).**
- **Verify with 3 assert cases.**

Expected Output #6:

- **Corrected code with type handling.**
- **AI explanation of the fix.**

Successful test validation.



```
1 def add_values():
2     return 5 + int("10")
3     print(add_values())
```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, !excludeText, t...) Code

[Running] python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCodeRunnerFile.python"

15

[Done] exited with code=0 in 0.381 seconds

Task 12 (Type Error – String + List Concatenation)

Task: Analyze code where a string is incorrectly added to a list.

Bug: Adding string and list

```
def combine():
    return "Numbers: " + [1, 2, 3]
print(combine())
```

Requirements:

- Run the code to observe the error.
- Explain why str + list is invalid.
- Fix using conversion (str([1,2,3]) or " ".join()).
- Verify with 3 assert cases.

Expected Output:

- Corrected code
- Explanation
- Successful test validation

```
tal(numbers): Untitled-14 • def calculate_area(length, width): Untitled-15 • def add_values(): Untitled-16 • def combine(): Untitled-17 •
1 def combine():
2     return "Numbers: " + str([1, 2, 3])
3 print(combine())

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, lexcludeText, t...) Code
[Running] python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCodeRunnerFile.python"
Numbers: [1, 2, 3]

[Done] exited with code=0 in 0.387 seconds
```

Task 13 (Type Error – Multiplying String by Float)

Task: Detect and fix code where a string is multiplied by a float.

Bug: Multiplying string by float

```
def repeat_text():
    return "Hello" * 2.5
print(repeat_text())
```

Requirements:

- **Observe the error.**
- **Explain why float multiplication is invalid for strings.**
- **Fix by converting float to int.**
- **Add 3 assert test cases.**

```
1 def repeat_text():
2     return "Hello" * 2
3     print(repeat_text())
```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, excludeText, t...) Code

[Running] python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCodeRunnerFile.python"

HelloHello

[Done] exited with code=0 in 0.424 seconds

Task 14 (Type Error – Adding None to Integer)

Task: Analyze code where None is added to an integer.

Bug: Adding None and integer

def compute():

value = None

return value + 10

print(compute())

Requirements:

- **Run and identify the error.**
- **Explain why NoneType cannot be added.**
- **Fix by assigning a default value.**
- **Validate using asserts.**

```
def add_values(): Untitled-16
def combine(): Untitled-17
def repeat_text(): Untitled-18
def compute(value): Untitled-19

1 def compute(value):
2     """Loading... lue + 10"""
3     print(compute(5))
```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, lexcludeText, t...) Code

[Running] python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCodeRunnerFile.python"

15

[Done] exited with code=0 in 0.41 seconds

Task 15 (Type Error – Input Treated as String Instead of Number)

Task: Fix code where user input is not converted properly.

Bug: Input remains string

```
def sum_two_numbers():
```

```
    a = input("Enter first number: ")
```

```
    b = input("Enter second number: ")
```

```
    return a + b
```

```
print(sum_two_numbers())
```

Requirements:

- Explain why input is always string.
- Fix using `int()` conversion.
- Verify with assert test cases.

```
combine(): Untitled-17 • def repeat_text(): Untitled-18 • def compute(value): Untitled-19
1 def sum_two_numbers():
2     a = input("Enter first number: ")
3     b = input("Enter second number: ")
4     return int(a) + int(b)
5 print(sum_two_numbers())
```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, lexer)

[Running] python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCo
Enter first number:
[Done] exited with code=1 in 11.434 seconds

[Running] python -u "c:\Users\naksh\OneDrive\Desktop\DEVOPS PROGRAMS\tempCo
Enter first number: